

# 1.Explain the Critical Section Problem. What are the three requirements that every solution to the Critical Section Problem must satisfy?

## Critical Section Problem:

A Critical Section is a part of a program in which a process accesses shared resources such as shared variables, files, memory, or tables. Since multiple processes may execute concurrently, only one process should execute in its critical section at a time to avoid inconsistency.

When one process is executing inside its critical section, no other process is allowed to execute its critical section.

## Structure of a Process:

Every process consists of four sections:

```
do
{
    Entry Section

    Critical Section

    Exit Section

    Remainder Section

} while(TRUE);
```

## Entry Section:

The process requests permission to enter the critical section.

## Critical Section:

The process accesses shared resources.

## Exit Section:

The process releases the shared resources.

## Remainder Section:

The remaining part of the program where no shared resources are accessed.

## Critical Section Problem-

The Critical Section Problem is the problem of designing a protocol so that multiple processes can share data safely without causing race conditions.

If two or more processes access shared data simultaneously, incorrect results may occur. Therefore, a proper synchronization mechanism is required.

### Requirements of a Solution to the Critical Section Problem

A correct solution must satisfy the following three requirements:

#### 1. Mutual Exclusion-

- If one process is executing in its critical section, no other process can execute its critical section simultaneously.
- This ensures that shared data is accessed by only one process at a time.

Example: If Process  $P_1$  is updating a shared file, Process  $P_2$  must wait until  $P_1$  finishes.

#### 2. Progress-

- If no process is executing in its critical section and some processes wish to enter, then only those processes that want to enter should participate in deciding which process enters next.
- The decision should not be postponed indefinitely.

This prevents unnecessary delays.

#### 3. Bounded Waiting-

- After a process requests to enter its critical section, there must be a limit on the number of times other processes are allowed to enter before its request is granted.
- This prevents starvation.

Every waiting process eventually gets a chance to execute.

### Approaches to Handle Critical Section

The operating system uses two general approaches:

#### 1. Preemptive Kernel-

- Allows a process running in kernel mode to be preempted.
- Another process may interrupt and execute.

#### 2. Non-Preemptive Kernel-

- Does not allow interruption while a process is executing in kernel mode.
- The process continues until it voluntarily exits the kernel.

### Applications:

The Critical Section Problem occurs whenever multiple processes access:

- Shared variables

- Shared files
- Shared memory
- Database records
- Shared devices

**2.Explain Peterson's Solution for the Critical Section Problem with algorithm and explain how it satisfies the three requirements.**

**Peterson's Solution:**

**Peterson's Solution is a software-based solution to the Critical Section Problem. It provides mutual exclusion between two processes without requiring any special hardware instructions.**

**It is applicable only for two processes, which alternate execution between their Critical Section and Remainder Section.**

**Shared Variables-**

**Peterson's solution requires two shared variables:**

**int turn;**

**boolean flag[2];**

**1. turn-**

- Indicates whose turn it is to enter the critical section.
- At any time, turn can be either 0 or 1.
- If  $\text{turn} = i$ , then process  $P_i$  is allowed to enter its critical section.
- If  $\text{turn} = j$ , then process  $P_j$  is allowed to enter its critical section.

**2. flag[ ]-**

- It indicates whether a process is ready to enter the critical section.
- If  $\text{flag}[i] = \text{true}$ , then process  $P_i$  wants to enter the critical section.

**Algorithm:**

**For process  $P_i$ :**

```

do
{
    flag[i] = true;
    turn = j;

    while(flag[j] && turn == j);

    /* Critical Section */

    flag[i] = false;

    /* Remainder Section */

} while(true);

```

Where,

- i represents the current process.
- j represents the other process.

### Working of Peterson's Solution:

1. Initially, process  $P_i$  sets  $flag[i] = true$ , indicating its intention to enter the critical section.
2. It then sets  $turn = j$ , giving the other process a chance to execute first.
3. The while loop checks:  $flag[j] == true \ \&\& \ turn == j$ 
  - If the other process also wants to enter ( $flag[j] = true$ ) and it is the other process's turn ( $turn = j$ ), then  $P_i$  waits.
  - Otherwise,  $P_i$  enters the critical section.
4. After completing the critical section,  $P_i$  sets:  $flag[i] = false$  - indicating that it no longer requires the critical section and proceeds to the remainder section.

### How Peterson's Solution Satisfies the Three Requirements

#### 1. Mutual Exclusion:

- A process enters its critical section only if:  $flag[j] == false \ OR \ turn == i$
- If both processes attempt to enter simultaneously, both flags become true, but turn can contain only one value (0 or 1).
- Therefore, only one process enters the critical section while the other waits.

Hence, Mutual Exclusion is satisfied.

#### 2. Progress:

- If the other process is not interested in entering the critical section ( $flag[j] = false$ ), the current process enters immediately.
- If both processes want to enter, the value of turn decides which process enters first.
- After one process completes its critical section, the waiting process gets the opportunity to execute.

Hence, Progress is satisfied.

#### 3. Bounded Waiting:

- Before re-entering the critical section, a process must set turn in favor of the other process.
- Thus, a waiting process cannot be postponed indefinitely.
- A process waits for at most one turn before entering the critical section.

Hence, Bounded Waiting is satisfied.

**Problem with Peterson's Solution:**

**Although Peterson's Solution satisfies all three requirements, it has certain limitations:**

- It works only for two processes.
- It assumes that machine instructions are executed in order.
- Modern computer architectures may reorder instructions, so Peterson's Solution does not always work correctly on modern multiprocessor systems.
- Disabling interrupts is feasible only in a single-processor environment and is inefficient in multiprocessor systems because interrupt messages must be sent to all processors, reducing system efficiency.

**Advantages**

- Software-based solution.
- No special hardware instructions required.
- Ensures:
  - Mutual Exclusion
  - Progress
  - Bounded Waiting

**3.Explain Hardware Solutions for Mutual Exclusion using Test-and-Set and Compare-and-Swap Instructions.**

**Hardware Solutions for Mutual Exclusion:**

Modern computer systems provide special hardware instructions that perform operations atomically (indivisibly). These instructions are used to solve the Critical Section Problem and ensure Mutual Exclusion.

Two hardware synchronization techniques are:

1. Test-and-Set (TAS)
2. Compare-and-Swap (CAS)

**1. Test-and-Set Instruction-**

The `test_and_set()` instruction is executed atomically. If two processes execute the instruction simultaneously, they are executed one after another in some arbitrary order.

A boolean variable `lock` is used and is initially set to false.

## Test-and-Set Function:

```
boolean test_and_set(boolean *target)
{
    boolean rv = *target;
    *target = true;
    return rv;
}
```

### Explanation

- Stores the current value of target in rv.
- Sets target = true.
- Returns the old value.

Since this operation is atomic, no two processes can modify the lock simultaneously.

```
boolean waiting[i] = false;
boolean lock = false;

do
{
    waiting[i] = true;
    key = true;

    while(waiting[i] && key)
        key = test_and_set(&lock);

    waiting[i] = false;

    /* Critical Section */

    j = (i + 1) % n;

    while((j != i) && !waiting[j])
        j = (j + 1) % n;

    if(j == i)
        lock = false;
    else
        waiting[j] = false;

    /* Remainder Section */
} while(true);
```

## How Test-and-Set Satisfies the Requirements:

### 1. Mutual Exclusion-

- The first process executing test\_and\_set() acquires the lock.
- Other processes must wait until the lock is released.
- Hence, only one process enters the critical section.

## 2. Progress-

- When a process exits the critical section, it either:
  - sets lock = false, or
  - allows another waiting process to proceed.
- Therefore, another waiting process can enter.

## 3. Bounded Waiting-

- Waiting processes are checked in cyclic order.
- Every waiting process gets a chance within n-1 turns.
- Hence, starvation is avoided.

## 2. Compare-and-Swap (CAS) Instruction:

The `compare_and_swap()` instruction is also an atomic instruction.

It compares the current value with an expected value. If both are equal, it replaces the value with a new value.

Compare-and-Swap Function:

```
int compare_and_swap
(int *value,
 int expected,
 int new_value)
{
    int temp = *value;

    if(*value == expected)
        *value = new_value;

    return temp;
}
```

Algorithm-

```
do
{
    while(compare_and_swap(&lock,0,1)!=0);

    /* Critical Section */

    lock = 0;

    /* Remainder Section */

} while(true);
```

## Working of Compare-and-Swap-

- A global variable lock is initialized to 0.

- The first process executes `compare_and_swap()`, changes `lock = 1`, and enters the critical section.
- Other processes fail because `lock ≠ 0`.
- After completing execution, the process resets `lock = 0`.
- The next waiting process can now enter the critical section.

### Problem with Hardware Solutions-

Although hardware instructions provide mutual exclusion, they have the following disadvantages:

- They are complex.
- They are not easily accessible to application programmers.
- They depend on special hardware support.

### Difference between Test-and-Set and Compare-and-Swap

| Test-and-Set                                               | Compare-and-Swap                                          |
|------------------------------------------------------------|-----------------------------------------------------------|
| Uses a boolean lock variable.                              | Uses a lock variable with expected and new values.        |
| Returns the previous value after setting the lock to true. | Replaces the value only if it matches the expected value. |
| Simpler implementation.                                    | More flexible and powerful.                               |
| Mainly used for simple mutual exclusion.                   | Used in advanced synchronization mechanisms.              |

## 4.Explain the Producer–Consumer (Bounded Buffer) Problem and its Solution using Semaphores.

**Producer–Consumer Problem (Bounded Buffer Problem):**

The Producer–Consumer Problem is a classical synchronization problem.

- A Producer Process produces information (items) and places them into a shared buffer.
- A Consumer Process removes and consumes the items from the same buffer.
- The buffer is shared between the producer and consumer processes.

The producer and consumer must be synchronized so that:

- The producer does not insert an item into a full buffer.
- The consumer does not remove an item from an empty buffer.



## Bounded Buffer:

The buffer has a fixed size (n).

- If the buffer is full, the producer must wait.
- If the buffer is empty, the consumer must wait.

This problem is known as the Bounded Buffer Problem.

## Solution Using Semaphores

The following variables are used:

```
int n;  
  
semaphore mutex = 1;  
semaphore empty = n;  
semaphore full = 0;
```

## Explanation of Variables

- n → Size of the buffer.
- mutex → Ensures mutual exclusion while accessing the buffer.
- empty → Counts the number of empty locations in the buffer.
- full → Counts the number of filled locations in the buffer.

Initially,

- empty = n
- full = 0

This means all buffer locations are empty.

## Producer Process

```
do  
{  
    /* Produce an item */  
  
    wait(empty);  
  
    wait(mutex);  
  
    /* Add item to buffer */  
  
    signal(mutex);  
  
    signal(full);  
  
} while(TRUE);
```

## Explanation of Producer Process:

### wait(empty)

- Checks whether an empty location is available.
- If the buffer is full, the producer waits.

### wait(mutex)

- Acquires the lock.
- Allows only one process to access the buffer.

### Add Item

- The producer inserts the produced item into the buffer.

### signal(mutex)

- Releases the lock.

### signal(full)

- Indicates that one more buffer location has become full.

## Consumer Process

```
do
{
    wait(full);

    wait(mutex);

    /* Remove item from buffer */

    signal(mutex);

    signal(empty);

    /* Consume the item */

} while(TRUE);
```

## Explanation of Consumer Process:

### wait(full)

- Checks whether the buffer contains any item.
- If the buffer is empty, the consumer waits.

### wait(mutex)

- Acquires the lock before accessing the buffer.

## Remove Item

- Removes an item from the buffer.

## signal(mutex)

- Releases the lock.

## signal(empty)

- Indicates that one more location in the buffer has become empty after consumption.

## Working of the Algorithm:

1. Initially, all buffer locations are empty.
2. The producer checks for an empty location using wait(empty).
3. After acquiring mutex, it inserts an item into the buffer.
4. It releases the buffer using signal(mutex) and increments full.
5. The consumer checks whether any item is available using wait(full).
6. It acquires mutex, removes the item, releases the lock, and increments empty.
7. Thus, producer and consumer execute safely without conflicts.

## Advantages:

- Prevents race conditions.
- Ensures mutual exclusion.
- Prevents the producer from writing into a full buffer.
- Prevents the consumer from reading from an empty buffer.
- Provides proper synchronization between producer and consumer processes.

## 5.Explain the Readers–Writers Problem and its Solution using Semaphores.

### Readers–Writers Problem:

The Readers–Writers Problem is a classical synchronization problem that occurs when a shared file/database is accessed by Reader and Writer processes.

- A Reader Process reads the data from the shared file.
- A Writer Process modifies (writes) the data in the shared file.

Since readers do not modify data, multiple readers can read simultaneously. However, if a writer is writing, no other reader or writer is allowed to access the shared file, otherwise data inconsistency may occur. Therefore, the writer must have exclusive access to the shared file.

### Solution Using Semaphores:

The solution uses the following variables:

```
semaphore rw_mutex = 1;
semaphore mutex = 1;
int read_count = 0;
```

### Explanation-

- `rw_mutex` → Controls access to the shared file.
- `mutex` → Protects the `read_count` variable.
- `read_count` → Stores the number of reader processes currently reading the file.

### Initially,

- `rw_mutex = 1`
- `mutex = 1`
- `read_count = 0`

### Reader Process:

```
do
{
    wait(mutex);

    read_count++;

    if(read_count == 1)
        wait(rw_mutex);

    signal(mutex);

    /* Reading is performed */

    wait(mutex);

    read_count--;

    if(read_count == 0)
        signal(rw_mutex);

    signal(mutex);

} while(TRUE);
```

### Explanation of Reader Process:

#### Step 1

#### `wait(mutex)`

- Allows only one reader to update `read_count`.

## Step 2

`read_count++`

- Increments the number of active readers.

## Step 3

`if(read_count == 1)`

`wait(rw_mutex)`

- If it is the first reader, it locks the shared file.
- This prevents any writer from entering while readers are reading.

## Step 4

`signal(mutex)`

- Releases the mutex lock.

## Step 5

Reading is performed

- Multiple readers can execute simultaneously.

## Step 6

`wait(mutex)`

- Again acquires mutex before updating `read_count`.

## Step 7

`read_count--`

- Decreases the number of readers.

## Step 8

`if(read_count == 0)`

`signal(rw_mutex)`

- If it is the last reader, it releases the shared file.
- Writers can now access the file.

## Step 9

`signal(mutex)`

- Releases the mutex.

**Writer Process:**

```
do
{
    wait(rw_mutex);

    /* Writing is performed */

    signal(rw_mutex);

} while(TRUE);
```

**Explanation of Writer Process-**

**wait(rw\_mutex)**

- Writer acquires exclusive access to the shared file.

**Writing**

- The writer modifies the contents of the shared file.

**signal(rw\_mutex)**

- Releases the file after writing is completed.

**Working of the Algorithm:**

1. Initially, read\_count = 0.
2. The first reader enters, increments read\_count, and locks rw\_mutex.
3. Other readers can enter and read simultaneously without blocking each other.
4. Writers must wait until all readers finish.
5. When the last reader exits, it releases rw\_mutex.
6. The writer then acquires rw\_mutex, writes the data, and releases it after completion.

**Advantages:**

- Allows multiple readers to read simultaneously.
- Ensures exclusive access for writers.
- Prevents data inconsistency.
- Provides proper synchronization between reader and writer processes.

**6.Explain the Dining Philosophers Problem and its Solution using Semaphores and Monitors.**

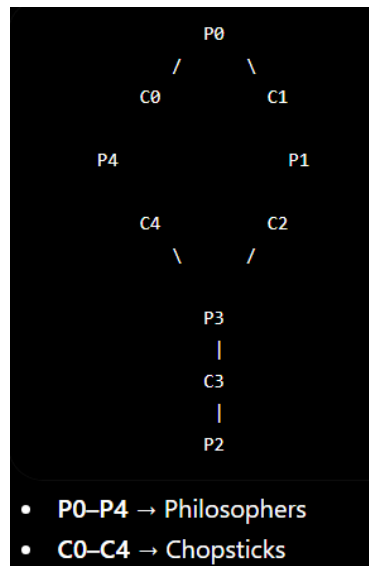
## Dining Philosophers Problem:

The Dining Philosophers Problem is one of the classical synchronization problems in Operating Systems. It illustrates the problems of process synchronization, resource sharing, and deadlock.

Consider five philosophers sitting around a circular table.

- There is one bowl of rice placed at the center of the table.
- Five chopsticks are placed between adjacent philosophers.
- A philosopher alternates between thinking and eating.
- To eat, a philosopher must acquire both the left and right chopsticks.
- A philosopher can pick up only one chopstick at a time.
- If a required chopstick is already in use by a neighboring philosopher, the philosopher must wait.

Diagram:



## Solution Using Semaphores

Each chopstick is represented by a semaphore.

## Algorithm

```

semaphore chopstick[5];

do
{
    wait(chopstick[i]);

    wait(chopstick[(i+1)%5]);

    /* Eat */

    signal(chopstick[i]);

    signal(chopstick[(i+1)%5]);

    /* Think */

} while(TRUE);

```

### Explanation

- Each chopstick is represented as a binary semaphore.
- Initially,

$\text{chopstick}[0\dots4] = 1$

- A philosopher executes `wait()` to acquire the left chopstick.
- Then executes `wait()` to acquire the right chopstick.
- After acquiring both chopsticks, the philosopher starts eating.
- After eating, both chopsticks are released using `signal()`.
- The philosopher then returns to thinking.

### Problem with Semaphore Solution:

Although this solution prevents two neighboring philosophers from eating simultaneously, it may lead to deadlock.

### Deadlock Situation-

Suppose all five philosophers become hungry at the same time.

1. Each philosopher picks up the left chopstick.
2. Every philosopher now waits for the right chopstick.
3. Since every right chopstick is already held by another philosopher, no philosopher can proceed.
4. All philosophers wait forever.

Hence, Deadlock occurs.

### Remedies to Deadlock



The PPT suggests the following methods:

1. Allow at most four philosophers to sit at the table simultaneously.
2. Allow a philosopher to pick up chopsticks only if both are available.
3. Use an Asymmetric Solution:
  - Odd-numbered philosophers pick up the left chopstick first and then the right.
  - Even-numbered philosophers pick up the right chopstick first and then the left.

**Solution Using Monitors-**

Monitors provide a deadlock-free solution.

A philosopher can eat only if both neighboring philosophers are not eating.

There are three possible states for each philosopher:

```
enum
{
    THINKING,
    HUNGRY,
    EATING
} state[5];
```

A condition variable is declared as:

`condition self[5];`

**Monitor Algorithm:**

```
monitor Dining_Philosophers
{
    enum {THINKING, HUNGRY, EATING} state[5];

    condition self[5];

    void pickup(int i)
    {
        state[i]=HUNGRY;
        test(i);

        if(state[i]!=EATING)
            self[i].wait();
    }

    void putdown(int i)
    {
        state[i]=THINKING;

        test((i+4)%5);

        test((i+1)%5);
    }
}
```

```

void test(int i)
{
    if(state[(i+4)%5]!=EATING &&
        state[i]==HUNGRY &&
        state[(i+1)%5]!=EATING)
    {
        state[i]=EATING;

        self[i].signal();
    }
}

```

## Working of Monitor Solution

1. When a philosopher becomes hungry, it calls pickup(i).
2. The monitor checks whether both neighboring philosophers are not eating.
3. If both chopsticks are available, the philosopher enters the EATING state.
4. Otherwise, the philosopher waits.
5. After eating, the philosopher calls putdown(i).
6. The monitor checks whether neighboring hungry philosophers can now eat.

Thus, the monitor controls the allocation of chopsticks and ensures proper synchronization.

## Advantages of Monitor Solution

- Ensures Mutual Exclusion.
- Prevents Deadlock.
- Prevents neighboring philosophers from eating simultaneously.
- Simplifies synchronization by automatically allowing only one process inside the monitor at a time.

## Difference between Semaphore and Monitor Solution

| Semaphore Solution                                       | Monitor Solution                                         |
|----------------------------------------------------------|----------------------------------------------------------|
| Uses binary semaphores for chopsticks.                   | Uses monitor and condition variables.                    |
| May lead to deadlock.                                    | Deadlock-free solution.                                  |
| Synchronization is managed explicitly by the programmer. | Synchronization is handled automatically by the monitor. |
| More prone to programming errors.                        | Safer and easier to implement.                           |

**7.Explain Semaphores in detail. Explain the types of Semaphores and the operations wait() and signal().**

## Semaphores

A Semaphore is a synchronization tool used to solve the Critical Section Problem and provide process synchronization.

A semaphore  $S$  is an integer variable that can be accessed only through two atomic operations:

- `wait()`
- `signal()`

Since these operations are atomic, only one process can modify the semaphore at a time, ensuring mutual exclusion.

### Semaphore Operations

#### 1. `wait()` Operation

```
wait(S)
{
    while(S <= 0);    // Busy waiting
    S--;
}
```

#### Explanation

- Checks whether the semaphore value is greater than zero.
- If  $S \leq 0$ , the process waits (busy waiting).
- Otherwise, the semaphore value is decremented by one ( $S--$ ) and the process enters the critical section.

#### 2. `signal()` Operation

```
signal(S)
{
    S++;
}
```

#### Explanation

- Increments the semaphore value ( $S++$ ).
- Indicates that the resource has been released.
- Allows another waiting process to enter the critical section.

### Semaphore Usage

Operating systems provide two types of semaphores:

1. Binary Semaphore
2. Counting Semaphore

### 1. Binary Semaphore:

A Binary Semaphore can take only two values:

- 0
- 1

It behaves similarly to a Mutex Lock.

Features-

- Used for Mutual Exclusion.
- Allows only one process to access the critical section at a time.
- Prevents race conditions.

### 2. Counting Semaphore:

A Counting Semaphore can take any non-negative integer value.

Features-

- Used when multiple instances of a resource are available.
- Initialized to the number of available resources.
- Each process performs wait() before using a resource.
- After releasing the resource, it performs signal().

When the semaphore value becomes 0, all resources are busy, and new processes must wait until a resource is released.

Semaphores for Process Synchronization-

Consider two processes:

- P1 executes statement S1
- P2 executes statement S2

Suppose S2 must execute only after S1.

A common semaphore synch is initialized to 0.

Process P1-

```
S1;  
signal(synch);
```

Process P2-

```
wait(synch);  
S2;
```

Since  $\text{synch} = 0$ , P2 waits until P1 executes `signal(synch)` after completing S1. Thus, synchronization between the two processes is achieved.

### Semaphore Implementation (Without Busy Waiting)

The basic `wait()` and `signal()` operations use busy waiting, which wastes CPU time.

**To overcome this problem:**

- If a process executes `wait()` and the semaphore value is not positive, the process is blocked and placed in a waiting queue.
- The CPU scheduler selects another process for execution.
- When another process executes `signal()`, one blocked process is removed from the waiting queue and awakened using `wakeup()`.

**Semaphore Structure:**

```
typedef struct  
{  
    int value;  
    struct process *list;  
} semaphore;
```

- `value` → Current semaphore value.
- `list` → Pointer to the waiting queue of blocked processes.

**Modified `wait()` Operation-**

```
wait(semaphore *S)  
{  
    S->value--;  
  
    if(S->value < 0)  
    {  
        add this process to S->list;  
        block();  
    }  
}
```

**Modified `signal()` Operation-**

```

signal(semaphore *S)
{
    S->value++;

    if(S->value <= 0)
    {
        remove a process P from S->list;
        wakeup(P);
    }
}

```

### Advantages of Semaphores-

- Provides Mutual Exclusion.
- Synchronizes concurrent processes.
- Prevents Race Conditions.
- Can be used for process synchronization and resource management.
- Supports both Binary and Counting semaphores.

### Problems with Semaphores-

If semaphores are used incorrectly, they may lead to:

- Busy Waiting
- Deadlock
- Starvation
- Programming errors, such as:
  - Interchanging wait() and signal()
  - Replacing signal() with wait()
  - Omitting wait() or signal() operations

These mistakes may violate mutual exclusion or cause deadlocks.

**8.Explain Monitors. How do Monitors overcome the problems of Semaphores?  
Explain the Dining Philosophers Solution using Monitors.**

### Monitors-

A Monitor is a high-level language synchronization construct developed to overcome the problems associated with semaphores.

A monitor is an Abstract Data Type (ADT) that contains:

- Shared variables
- Programmer-defined operations (functions)
- Initialization code

The monitor provides automatic mutual exclusion, ensuring that only one process can execute inside the monitor at a time.

## Monitor Syntax

```
monitor monitor_name
{
    /* Shared Variable Declarations */

    function P1(...)
    {
        ...
    }

    function P2(...)
    {
        ...
    }

    ...

    function Pn(...)
    {
        ...
    }

    initialization_code(...)
    {
        ...
    }
}
```

## Features of Monitors

- Only one process can execute inside a monitor at a time.
- Shared variables can be accessed only by monitor functions.
- External processes cannot directly access monitor variables.
- The programmer need not explicitly write synchronization code because the monitor automatically provides Mutual Exclusion.

## Condition Variables:

To solve synchronization problems, monitors use condition variables.

### Syntax-

condition x, y;

The two operations performed on condition variables are:

#### 1. wait()

x.wait()

- Suspends the calling process.
- The process remains blocked until another process executes x.signal().

## 2. signal()

x.signal()

- Wakes up exactly one waiting process.

### Dining Philosophers Solution Using Monitors

The monitor provides a deadlock-free solution to the Dining Philosophers Problem.

A philosopher is allowed to eat only if both neighboring philosophers are not eating.

Philosopher States:

Each philosopher can be in one of the following three states:

```
enum
{
    THINKING,
    HUNGRY,
    EATING
} state[5];
```

A condition variable is declared as:

condition self[5];

Monitor Algorithm:

monitor Dining\_Philosophers:

```
monitor Dining_Philosophers
{
    enum {THINKING,HUNGRY,EATING} state[5];

    condition self[5];

    void pickup(int i)
    {
        state[i]=HUNGRY;

        test(i);

        if(state[i]!=EATING)
            self[i].wait();
    }

    void putdown(int i)
    {
        state[i]=THINKING;

        test((i+4)%5);

        test((i+1)%5);
    }
}
```



```

void test(int i)
{
    if(state[(i+4)%5]!=EATING &&
        state[i]==HUNGRY &&
        state[(i+1)%5]!=EATING)
    {
        state[i]=EATING;

        self[i].signal();
    }
}

initialization_code()
{
    for(int i=0;i<5;i++)
        state[i]=THINKING;
}
}

```

## Working of the Monitor Solution

### Step 1

When a philosopher becomes hungry, it calls:

`pickup(i)`

The philosopher's state changes from THINKING to HUNGRY.

### Step 2

The `test(i)` function checks whether:

- Left neighbor is not eating.
- Right neighbor is not eating.

If both conditions are true, the philosopher enters the EATING state.

Otherwise, it waits.

### Step 3

After completing the meal, the philosopher calls:

`putdown(i)`

Its state changes to THINKING.

### Step 4

The monitor then checks whether neighboring hungry philosophers can now eat.

If possible, one waiting philosopher is awakened using:

```
self[i].signal()
```

Thus, philosophers eat one after another without conflict.

### How Monitors Overcome the Problems of Semaphores:

Semaphores may lead to:

- Busy Waiting
- Deadlock
- Programming errors
- Incorrect ordering of wait() and signal()

Monitors overcome these problems because:

- Mutual exclusion is automatic.
- Shared variables are protected inside the monitor.
- Condition variables manage waiting and signaling safely.
- Only one process can execute inside the monitor at any time.
- The Dining Philosophers solution using monitors is deadlock-free.

### Advantages of Monitors

- High-level synchronization mechanism.
- Automatic Mutual Exclusion.
- Easy to program.
- Eliminates many semaphore programming errors.
- Provides deadlock-free solution for the Dining Philosophers Problem.
- Improves reliability and readability of synchronization code.

## 9.Explain Deadlocks. Explain the Necessary Conditions and Methods for Handling Deadlocks.

### Deadlock:

A Deadlock is a situation in which a set of processes is permanently blocked because each process is waiting for an event (resource release) that can be caused only by another process in the same set. As a result, none of the processes can proceed with execution.

### Example of Deadlock

Consider two processes  $P_0$  and  $P_1$  and two semaphores S and Q.

Initially,

$S = 1$

$Q = 1$

Process  $P_0$ -

```
wait(S);  
wait(Q);  
  
...  
  
signal(S);  
signal(Q);
```

Process  $P_1$ :

```
wait(Q);  
wait(S);  
  
...  
  
signal(Q);  
signal(S);
```

Working:

- Process  $P_0$  acquires semaphore S.
- Process  $P_1$  acquires semaphore Q.
- Now  $P_0$  waits for Q, which is held by  $P_1$ .
- At the same time,  $P_1$  waits for S, which is held by  $P_0$ .
- Since neither process can proceed, both remain blocked forever.

Hence, a Deadlock occurs.

### Necessary Conditions for Deadlock (Coffman Conditions)

A deadlock can occur only if all four of the following conditions hold simultaneously.

#### 1. Mutual Exclusion

- At least one resource must be non-shareable.
- Only one process can use the resource at a time.

#### 2. Hold and Wait

- A process holding one or more resources waits to acquire additional resources held by other processes.

### 3. No Preemption

- Resources cannot be forcibly taken away.
- A resource is released only voluntarily by the process after completing its task.

### 4. Circular Wait

- A circular chain of waiting processes exists.
- Each process waits for a resource held by the next process in the cycle.

If all four conditions are present simultaneously, a deadlock occurs.

### Methods for Handling Deadlocks:

The operating system can handle deadlocks using the following methods:

#### 1. Deadlock Prevention

- Prevent deadlock by ensuring that at least one of the four necessary conditions does not occur.
- Resources are allocated in such a way that deadlock cannot happen.

#### 2. Deadlock Avoidance

- The operating system allocates resources only if the system remains in a safe state.
- The Banker's Algorithm is commonly used for deadlock avoidance.

#### 3. Deadlock Detection

- Allow deadlocks to occur.
- Periodically check the system to determine whether a deadlock has occurred.

#### 4. Deadlock Recovery

If a deadlock is detected, the operating system recovers by:

- Terminating one or more processes.
- Preempting resources from selected processes.
- Rolling back processes to a safe state.

These methods remove the deadlock and allow execution to continue.

### Effects of Deadlock

- Processes remain blocked indefinitely.
- CPU utilization decreases.
- System throughput decreases.
- Resources remain occupied without useful work.
- Overall system performance is affected.

## 10.Explain Deadlock Prevention.

## Deadlock Prevention:

Deadlock Prevention is a method of handling deadlocks by ensuring that at least one of the four necessary conditions for deadlock never holds. By violating any one of these conditions, the operating system prevents deadlocks from occurring.

The four necessary conditions are:

1. Mutual Exclusion
2. Hold and Wait
3. No Preemption
4. Circular Wait

Deadlock prevention works by eliminating one or more of these conditions.

### Methods of Deadlock Prevention

#### 1. Preventing Mutual Exclusion

##### Method

- Make resources sharable whenever possible.
- Allow multiple processes to use the same resource simultaneously.

##### Limitation

- This method cannot be applied to non-sharable resources such as printers and other I/O devices.

#### 2. Preventing Hold and Wait

##### Method

- A process must request all the resources it needs at one time before execution begins.
- Alternatively, a process must release all currently held resources before requesting additional resources.

##### Advantage

- Eliminates the possibility of a process holding one resource while waiting for another.

##### Disadvantage

- Resources may remain allocated even when they are not being used, resulting in poor resource utilization.

#### 3. Preventing No Preemption

##### Method

- If a process requests a resource that is unavailable, the operating system preempts (takes back) the resources currently held by that process.
- The process can continue only after all required resources become available.

#### Advantage

- Prevents indefinite waiting.

#### Limitation

- This method is applicable only to resources whose state can be safely saved and restored.

### 4. Preventing Circular Wait

#### Method

- Assign a unique ordering (number) to every resource type.
- Processes must request resources only in increasing order of their assigned numbers.

#### Advantage

- Since all processes follow the same resource order, a circular chain of waiting cannot occur.

### Advantages of Deadlock Prevention

- Completely prevents deadlocks.
- Ensures smooth execution of processes.
- Improves system reliability.
- Eliminates indefinite waiting due to deadlock.

### Disadvantages of Deadlock Prevention

- Leads to low resource utilization.
- Reduces system throughput.
- Some prevention methods are not practical for all types of resources.
- Processes may have to wait unnecessarily even when deadlock is unlikely.

## 11.Explain Deadlock Avoidance using Banker's Algorithm.

### Deadlock Avoidance:

Deadlock Avoidance is a method of handling deadlocks in which the operating system allocates resources only if the system remains in a safe state after allocation.

Before allocating resources, the operating system checks whether the requested allocation can be made without leading to a deadlock. If the allocation may result in an unsafe state, the requesting process must wait.

## Safe State:

A system is said to be in a Safe State if there exists at least one sequence in which all the processes can complete successfully without causing a deadlock.

This sequence is called a Safe Sequence.

If no such sequence exists, the system is in an Unsafe State. An unsafe state may eventually lead to a deadlock.

## Banker's Algorithm:

The Banker's Algorithm is a deadlock avoidance algorithm proposed by Edsger Dijkstra.

It is called the Banker's Algorithm because it works like a banker who grants loans only when there are sufficient resources to satisfy all customers safely.

The algorithm checks whether granting a resource request keeps the system in a safe state. If yes, the request is granted; otherwise, the requesting process must wait.

## Data Structures Used-

The Banker's Algorithm maintains the following data structures:

### 1. Available

- Indicates the number of available instances of each resource.

### 2. Max

- Represents the maximum number of resources each process may require.

### 3. Allocation

- Represents the number of resources currently allocated to each process.

### 4. Need

The remaining resource requirement of each process.

## Formula

$$\text{Need} = \text{Max} - \text{Allocation}$$

## Steps of Banker's Algorithm:

### Step 1

Calculate the Need matrix using:

$$\text{Need} = \text{Max} - \text{Allocation}$$

## Step 2

Find a process whose  $\text{Need} \leq \text{Available}$ .

## Step 3

If such a process is found:

- Execute the process.
- Release its allocated resources.
- Add the released resources to Available.

$\text{Available} = \text{Available} + \text{Allocation}$

## Step 4

Repeat Steps 2 and 3 until all processes finish.

## Step 5

- If all processes complete successfully, the system is in a Safe State.
- Otherwise, the system is in an Unsafe State, and the request is denied.

## Algorithm (Safety Algorithm)

1. Compute  $\text{Need} = \text{Max} - \text{Allocation}$ .
2. Set  $\text{Work} = \text{Available}$ .
3. Set  $\text{Finish}[i] = \text{false}$  for all processes.
4. Find a process  $P_i$  such that:
  - $\text{Finish}[i] = \text{false}$
  - $\text{Need}[i] \leq \text{Work}$
5. If found:
  - $\text{Work} = \text{Work} + \text{Allocation}[i]$
  - $\text{Finish}[i] = \text{true}$
  - Repeat Step 4.
6. If all  $\text{Finish}[i] = \text{true}$ , the system is in a Safe State.
7. Otherwise, the system is in an Unsafe State.

## Advantages of Banker's Algorithm

- Prevents deadlocks.
- Allocates resources safely.
- Ensures the system always remains in a Safe State.
- Suitable for systems where the maximum resource requirement is known in advance.

## Disadvantages of Banker's Algorithm

- Maximum resource requirements must be known beforehand.
- Not suitable for dynamic systems where resource requirements change frequently.
- Involves additional computation before every resource allocation.



- May reduce system performance due to frequent safety checks.

## 12.Explain Deadlock Detection and Recovery.

### Deadlock Detection:

Deadlock Detection is a method in which the operating system allows deadlocks to occur and then periodically checks the system to determine whether a deadlock has occurred. If a deadlock is detected, appropriate recovery methods are applied to restore normal system execution.

### Need for Deadlock Detection:

- Deadlocks are allowed to occur.
- The operating system continuously or periodically examines the resource allocation status.
- If a deadlock is found, suitable recovery techniques are applied.

### Detection Procedure:

The operating system performs the following steps:

1. Monitor the allocation of resources to processes.
2. Check whether processes are waiting indefinitely for resources.
3. Identify the set of deadlocked processes.
4. Initiate suitable recovery techniques.

### Deadlock Recovery-

Once a deadlock is detected, the operating system must recover from it so that the blocked processes can continue execution.

#### 1. Process Termination:

The operating system terminates one or more deadlocked processes to break the circular wait.

There are two approaches:

##### (a) Abort All Deadlocked Processes

- Immediately terminates every deadlocked process.
- Releases all occupied resources.
- Simple but causes maximum loss of work.

##### (b) Abort One Process at a Time

- Terminates one deadlocked process.
- Checks whether the deadlock has been removed.
- Repeats until the deadlock is eliminated.

This approach reduces the loss of completed work.

## Factors for Selecting a Process to Terminate

The operating system may consider:

- Priority of the process.
- Amount of computation completed.
- Number of resources held.
- Number of resources still required.
- Cost of terminating the process.

## 2. Resource Preemption

Instead of terminating processes, the operating system may preempt (take back) resources from selected processes.

The preempted resources are allocated to other processes so that execution can continue.

Resource preemption involves three steps:

### (a) Selecting a Victim

Choose the process from which resources should be taken.

### (b) Resource Preemption

Take the required resources from the selected process and allocate them to another process.

### (c) Rollback

The preempted process is rolled back to a previously saved safe state and restarted later.

This prevents loss of all completed work.

## Advantages of Deadlock Detection and Recovery

- Better resource utilization than prevention.
- Resources are allocated more freely.
- Deadlocks are handled only when they actually occur.
- Suitable when deadlocks are infrequent.

## Disadvantages

- Deadlock detection requires periodic checking.
- Recovery procedures increase system overhead.
- Process termination may result in loss of completed work.
- Resource preemption and rollback are complex to implement.

## Difference between Detection and Recovery

| Deadlock Detection                          | Deadlock Recovery                                |
|---------------------------------------------|--------------------------------------------------|
| Identifies whether a deadlock has occurred. | Removes the detected deadlock.                   |
| Continuously monitors the system.           | Uses process termination or resource preemption. |
| Detects deadlocked processes.               | Restores the system to normal execution.         |
| Performed before recovery.                  | Performed after detection.                       |

## 14.Explain Resource Allocation Graph (RAG). Explain the different types of Resource Allocation Graphs with neat diagrams.

### Resource Allocation Graph (RAG):

A Resource Allocation Graph (RAG) is a directed graph used to represent the allocation of resources to processes and the resource requests made by processes. It is used to identify and analyze deadlocks in an operating system.

### Components of Resource Allocation Graph

A Resource Allocation Graph is represented as:[ $G=(V,E)$ ]

where:

- V = Set of vertices
- E = Set of edges

The vertices are divided into two types.

### 1. Process Nodes

- Represent active processes.
- Denoted by circles.
- Example:(P1) (P2) (P3)

### 2. Resource Nodes

- Represent resource types.
- Denoted by rectangles.
- Each dot inside the rectangle represents one instance of that resource.

Example:



## Types of Edges

### 1. Request Edge: $P_i \rightarrow R_j$

- Indicates that Process  $P_i$  is requesting Resource  $R_j$ .
- The process must wait until the resource becomes available.

### 2. Assignment Edge: $R_j \rightarrow P_i$

- Indicates that an instance of Resource  $R_j$  has been allocated to Process  $P_i$ .

## Working of Resource Allocation Graph

- When a process requests a resource, a Request Edge is added.
- When the resource is allocated, the Request Edge is replaced by an Assignment Edge.
- When the process releases the resource, the Assignment Edge is removed.

## Types of Resource Allocation Graphs

### 1. Resource Allocation Graph without Deadlock

#### Characteristics:

- No cycle exists.
- All processes eventually obtain resources.
- No deadlock occurs.

#### Diagram:

$(P1) \rightarrow [R1]$

$[R2] \rightarrow (P1)$

$[R1] \rightarrow (P2)$

$(P2) \rightarrow [R3]$

$[R3] \rightarrow (P3)$

Since no cycle exists, the system is deadlock-free.

### 2. Resource Allocation Graph with Cycle and Deadlock

#### Example cycle:

P1 → R1 → P2 → R3 → P3 → R2 → P1

Working:

- P1 waits for R1.
- P2 waits for R3.
- P3 waits for R2.
- Each required resource is held by another process.

Hence, all processes wait forever.

Deadlock occurs.

### 3. Resource Allocation Graph with Cycle but No Deadlock

Sometimes a graph contains a cycle but no deadlock.

Reason:

- Resource types have multiple instances.
- Another process may release a resource and break the cycle.

Example:

P1 → R1 → P3 → R2 → P1

Although a cycle exists, another process may release R2, allowing execution to continue.

Therefore,

Cycle ≠ Deadlock when resources have multiple instances.

### Important Notes

- Single instance of each resource: A cycle always implies deadlock.
- Multiple instances of resources: A cycle does not necessarily imply deadlock.

### Applications

- Detecting deadlocks.
- Analyzing resource allocation.
- Understanding process-resource relationships.
- Designing deadlock detection algorithms.

### LAQ-15: Explain Deadlock Detection Algorithm.

Deadlock Detection Algorithm:

If a system does not use Deadlock Prevention or Deadlock Avoidance, deadlocks may occur. In such systems, the operating system periodically executes a Deadlock Detection

Algorithm to determine whether a deadlock exists. If a deadlock is detected, recovery methods are applied.

## 1. Detection for Single Instance of Each Resource

When each resource type has only one instance, a Wait-for Graph is used.

### Wait-for Graph

A Wait-for Graph is obtained from the Resource Allocation Graph by:

- Removing all resource nodes.
- Connecting one process directly to another.

If  $P_i \rightarrow P_j$  exists, it means:

$P_i$  is waiting for  $P_j$  to release a resource.

### Deadlock Rule

- If the Wait-for Graph contains a cycle, the system is deadlocked.
- If no cycle exists, no deadlock exists.

## 2. Detection for Multiple Instances of Resources

For systems having multiple instances of resources, the operating system uses the following data structures.

### Available

Number of available instances of each resource.

### Allocation Matrix

Resources currently allocated to each process.

### Request Matrix

Resources currently requested by each process.

### Work Vector

Initially,

Work = Available

### Finish Array

Initially,

- Finish[i] = False (if process holds resources)

- **Finish[i] = True (otherwise)**

## **Deadlock Detection Algorithm**

### **Step 1**

**Initialize**

**Work = Available**

**Initialize**

**Finish[i]**

**for every process.**

### **Step 2**

**Find a process satisfying:**

- **Finish[i] = False**
- **Request[i] ≤ Work**

### **Step 3**

**If found,**

**Work = Work + Allocation[i]**

**Set**

**Finish[i] = True**

**Repeat Step 2.**

### **Step 4**

**If every Finish[i] becomes True,**

**No Deadlock exists.**

**Otherwise,**

**Processes whose Finish[i] remains False are Deadlocked Processes.**

## **Complexity**

- **Wait-for Graph Algorithm:[ $O(n^2)$ ]**
- **Detection Algorithm (Multiple Resources):[ $O(m * n^2)$ ]**

**where:**

- **m = Number of resource types**
- **n = Number of processes**

### Example

The PPT gives an example with:

- Processes:  $P_0, P_1, P_2, P_3, P_4$
- Resources: A, B, C

Applying the detection algorithm gives the sequence:

$\langle P_0, P_2, P_3, P_1, P_4 \rangle$

Since all processes finish successfully, the system is not in a deadlock state.

### Advantages

- Detects deadlocks after they occur.
- Suitable when deadlocks are rare.
- Better resource utilization than prevention.

### Disadvantages

- Requires periodic execution.
- Detection introduces system overhead.
- Recovery is needed after detection.



## SAQ's:

### 1. What is Inter-Process Communication (IPC)?

Inter-Process Communication (IPC) is a mechanism that allows cooperating processes to exchange data and synchronize their activities.

#### Reasons for IPC:

- Information Sharing
- Computation Speedup
- Modularity
- Convenience

#### IPC Models:

- Shared Memory
- Message Passing



## 2. Explain the two IPC models.

### (a) Shared Memory

- Processes communicate by sharing a common memory area.
- Processes read and write data directly into shared memory.
- Fast communication.
- Processes themselves are responsible for synchronization.

### (b) Message Passing

- Processes exchange messages using send() and receive() operations.
- Suitable for distributed systems.
- No shared address space is required.

## 3. Strengths and Weaknesses of Shared Memory and Message Passing

| Shared Memory            | Message Passing                     |
|--------------------------|-------------------------------------|
| Faster communication     | Slower due to message transfer      |
| Suitable for same system | Suitable for distributed systems    |
| Requires synchronization | Synchronization is built-in         |
| High efficiency          | More secure and easier to implement |

## 4. What is a Critical Section?

A Critical Section is the part of a program where a process accesses shared resources such as shared variables, files or memory.

Only one process should execute in its critical section at a time to prevent inconsistency.

## 5. Define Race Condition.

A Race Condition occurs when two or more processes access and modify shared data simultaneously, and the final result depends on the order in which they execute.

It may produce incorrect or inconsistent results.

## 6. State the three requirements of the Critical Section Problem.

A correct solution must satisfy:

1. **Mutual Exclusion** – Only one process executes in the critical section at a time.
2. **Progress** – Waiting processes should not be postponed indefinitely.

3. **Bounded Waiting** – Every waiting process gets a chance to execute within a finite time.

## 7. Explain Test-and-Set instruction.

Test-and-Set is an atomic hardware instruction used to achieve mutual exclusion.

It tests the lock variable and sets it to true in a single indivisible operation, allowing only one process to enter the critical section at a time.

## 8. Explain Compare-and-Swap instruction.

Compare-and-Swap (CAS) is an atomic hardware instruction.

It compares the current value with an expected value. If both are equal, it replaces the value with a new value; otherwise, it leaves the value unchanged.

It is used to implement mutual exclusion.

## 9. What is Mutual Exclusion?

Mutual Exclusion is a synchronization technique that ensures only one process at a time can execute its critical section and access shared resources.

It prevents race conditions and maintains data consistency.

## 10. What are the classical synchronization problems?

The three classical synchronization problems are:

1. **Producer–Consumer (Bounded Buffer) Problem**
2. **Readers–Writers Problem**
3. **Dining Philosophers Problem**

## 11. What is the Producer–Consumer Problem?

The Producer–Consumer Problem is a synchronization problem in which:

- The Producer places items into a shared buffer.
- The Consumer removes items from the buffer.

The producer waits if the buffer is full, and the consumer waits if the buffer is empty.

**12. Explain the Readers–Writers Problem.** The Readers–Writers Problem involves a shared file accessed by readers and writers.

- Multiple readers can read simultaneously.
- Only one writer can write at a time.
- While writing, no reader or another writer is allowed to access the file.

## 13. Explain the Dining Philosophers Problem.

The Dining Philosophers Problem is a synchronization problem in which five philosophers share five chopsticks.

A philosopher needs both left and right chopsticks to eat. If every philosopher picks up one chopstick simultaneously, a deadlock may occur.

#### 14. What is a Semaphore?

A Semaphore is an integer synchronization variable accessed only through the atomic operations:

- wait()
- signal()

It is used to achieve mutual exclusion and process synchronization.

#### 15. Types of Semaphores.

There are two types:

1. Binary Semaphore
  - Values are 0 or 1.
  - Used for mutual exclusion.
2. Counting Semaphore
  - Can have multiple integer values.
  - Used when multiple instances of a resource are available.

#### 16. Explain wait() and signal() operations.

wait(S)

- Checks the semaphore value.
- If  $S \leq 0$ , the process waits.
- Otherwise, decrements the semaphore.

signal(S)

- Increments the semaphore.
- Releases the resource for another waiting process.

#### 17. What is a Monitor?

A Monitor is a high-level synchronization construct that provides automatic mutual exclusion.

It contains shared variables, functions, and initialization code. Only one process can execute inside a monitor at a time.

#### 18. Difference between Semaphore and Monitor.

|                                           |                                             |
|-------------------------------------------|---------------------------------------------|
| <b>Semaphore</b>                          | <b>Monitor</b>                              |
| <b>Low-level synchronization tool</b>     | <b>High-level synchronization construct</b> |
| <b>Uses wait() and signal()</b>           | <b>Uses condition variables</b>             |
| <b>Programmer manages synchronization</b> | <b>Automatic mutual exclusion</b>           |
| <b>More prone to programming errors</b>   | <b>Easier and safer to use</b>              |

### 19. What is Deadlock?

**A Deadlock is a condition in which two or more processes wait indefinitely for resources held by each other, so none of them can proceed.**

### 20. State the four necessary conditions for Deadlock.

- 1. Mutual Exclusion**
- 2. Hold and Wait**
- 3. No Preemption**
- 4. Circular Wait**

### 21. Define Safe State.

**A Safe State is a state in which there exists at least one sequence of process execution (safe sequence) such that every process can complete without causing a deadlock.**

### 22. What is a Resource Allocation Graph (RAG)?

**A Resource Allocation Graph (RAG) is a directed graph used to represent the allocation of resources to processes and the requests made by processes for resources.**

**It is used to analyze and detect deadlocks.**

### 23. Difference between Deadlock Prevention and Deadlock Avoidance.

| Deadlock Prevention                                    | Deadlock Avoidance                                      |
|--------------------------------------------------------|---------------------------------------------------------|
| Prevents deadlock by violating one necessary condition | Prevents deadlock by keeping the system in a safe state |
| Simpler approach                                       | Uses Banker's Algorithm                                 |
| Lower resource utilization                             | Better resource utilization                             |
| Less flexible                                          | More flexible                                           |

## 24. Difference between Safe State and Unsafe State.

| Safe State                        | Unsafe State                 |
|-----------------------------------|------------------------------|
| At least one safe sequence exists | No safe sequence exists      |
| Deadlock will not occur           | May lead to deadlock         |
| Resources can be allocated safely | Resource allocation is risky |

\*\*\*\*\*

## 1. Message Passing (SAQ)

### Message Passing:

**Message Passing is an Inter-Process Communication (IPC) mechanism in which cooperating processes communicate by sending and receiving messages without sharing the same address space.**

It is especially useful in distributed systems, where communicating processes may reside on different computers connected through a network. A message-passing system provides two operations:

- `send()`
- `receive()`

### Types of Communication

#### 1. Direct Communication

- **Sender explicitly names the receiver.**
- **Receiver explicitly names the sender.**

- **Operations:**
  - **send(P, message)**
  - **receive(Q, message)**

#### Properties:

- Link is automatically established.
- Link exists between exactly two processes.
- Only one communication link exists between a pair of processes.

## 2. Indirect Communication

- **Communication takes place through mailboxes (ports).**
- **Operations:**
  - **send(A, message)**
  - **receive(A, message)**

#### Properties:

- Communication is established only through a shared mailbox.
- More than two processes can share the same mailbox.
- Multiple communication links may exist.

### Synchronization in Message Passing

#### Message passing can be:

- Synchronous (Blocking)
- Asynchronous (Non-blocking)

#### Blocking Operations

- **Blocking Send:** Sender waits until the message is received.
- **Blocking Receive:** Receiver waits until a message arrives.

#### Non-blocking Operations

- **Non-blocking Send:** Sender continues execution after sending.
- **Non-blocking Receive:** Receiver continues even if no message is available.

#### Buffering

Messages are stored in temporary queues.

#### Three types of buffering:

1. **Zero Capacity** – No buffering; sender waits until the receiver receives the message.
2. **Bounded Capacity** – Queue has finite size; sender waits only when the queue is full.
3. **Unbounded Capacity** – Queue size is unlimited; sender never waits.

#### Advantages

- No shared memory required.
- Suitable for distributed systems.
- Provides both communication and synchronization.

## 2.What are Event Counters?

### Event Counters

An Event Counter is a synchronization mechanism used to coordinate the execution of concurrent processes by counting the occurrence of events.

Instead of controlling access to resources like semaphores, an event counter keeps track of how many times a particular event has occurred. A process can wait until the required event count is reached before continuing execution.

### Working

- Initially, the event counter is set to 0.
- Whenever a specific event occurs, the counter is incremented.
- Processes waiting for that event are resumed when the counter reaches the required value.

### Operations

1. **Advance (EC)**
  - Increments the event counter whenever an event occurs.
2. **Read (EC)**
  - Returns the current value of the event counter.
3. **Await (EC, value)**
  - Suspends the process until the event counter reaches the specified value.

### Advantages

- Provides process synchronization.
- Eliminates busy waiting.
- Simple mechanism for ordering events between concurrent processes.
- Useful when multiple processes must wait for the completion of specific events.

### Applications

- Process synchronization.
- Producer–Consumer synchronization.
- Multiprocessor systems.
- Operating system event handling.

\*\*\*\*\*

Banker's Algorithm problem:

## Step 1: Draw the Tables

Every question gives you:

- Allocation Matrix
- Max Matrix
- Available Vector

Step 2: Calculate the Need Matrix:  $\text{Need} = \text{Max} - \text{Allocation}$

Step 3: Write Work = Available (Think of Work as the resources currently available)

Step 4: Compare Need with Work ( $\text{Need} \leq \text{Work} ?$ )-if its true

process is executed, and new work is calculated with -  $[\text{new work} = \text{old work} + \text{allocation}(\text{of that process})]$ , and safe sequence is obtained

if its false it is not executed, and moved to next process....this continues till the final safe sequence is obtained (all processes are included in safe sequence)